

Practical No. 1

1) Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how Python accepts data from the user and displays the required information on the screen.

Source Code:

```
name = input("Enter your name: ")
age = input("Enter your age: ")

print("Hello", name)
print("Your age is", age)
```

Output:

```
Enter your name: Meet
Enter your age: 20
Hello Meet
Your age is 20
```

Conclusion:

Thus, the program successfully demonstrates how to take input from the user and display output using Python.

2) Definition : Write a program to illustrate the use of different data types and type casting.

Aim/ Objective: To understand the use of different data types in Python and learn how to convert one data type into another using type casting.

Source Code:

```
a = 10
b = 10.5
c = "20"
d = True
```

```
print("Integer:", a)
print("Float:", b)
print("String:", c)
print("Boolean:", d)

# Type casting
x = int(c)
y = float(a)
z = str(a)
print("String to Integer:", x)
print("Integer to Float:", y)
print("Integer to String:", z)
```

Output:

```
Integer: 10
Float: 10.5
String: 20
Boolean: True
String to Integer: 20
Integer to Float: 10.0
Integer to String: 10
```

Conclusion:

Thus, the program demonstrates different data types in Python and shows how type casting can be used to convert values from one data type to another.

3) Definition: Write a program to perform arithmetic relational and logical operations using Python operators...

Aim/ Objective:

To study and understand the different types of operators available in Python and to perform arithmetic, relational, and logical operations on given values using these operators.

Source Code:

```
a = 10
```

```
b = 5
```

```
# Arithmetic operators
```

```
print("Addition:", a + b)
```

```
print("Subtraction:", a - b)
```

```
print("Multiplication:", a * b)
```

```
print("Division:", a / b)
```

```
print("Modulus:", a % b)
```

```
# Relational operators
```

```
print("a is greater than b:", a > b)
```

```
print("a is equal to b:", a == b)
```

```
print("a is less than b:", a < b)
```

```
# Logical operators
```

```
print("a > b and b > 0:", a > b and b > 0)
```

```
print("a < b or b > 0:", a < b or b > 0)
```

```
print("not(a < b):", not(a < b))
```

Output:

Addition: 15

Subtraction: 5

Multiplication: 50

Division: 2.0

Modulus: 0

a is greater than b: True

a is equal to b: False

a is less than b: False

a > b and b > 0: True

a < b or b > 0: True

not(a < b): True

Conclusion:

Thus, the program successfully demonstrates the use of arithmetic, relational, and logical operators in Python and helps in understanding how these operators are used to perform different types of operations.

4) Definition : Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/ Objective:

To study and understand different string operations in Python, including string slicing, string formatting, and the use of various built-in string functions for manipulating and processing text.

Source Code:

```
text = "Python Programming"

# Slicing

print("Original String:", text)
print("First 6 characters:", text[:6])
print("Last 11 characters:", text[7:])


# String formatting

name = "Meet"
age = 20

print("My name is {} and I am {} years old.".format(name, age))
```

```
# Built-in string functions
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("Length:", len(text))
print("Replace:", text.replace("Python", "Java"))
print("Count of 'm':", text.count("m"))
```

Output:

Original String: Python Programming

First 6 characters: Python

Last 11 characters: Programming

My name is Meet and I am 20 years old.

Uppercase: PYTHON PROGRAMMING

Lowercase: python programming

Length: 18

Replace: Java Programming

Count of 'm': 2

Conclusion:

Thus, the program successfully demonstrates string slicing, formatting, and different built-in string functions in Python. These operations are useful for accessing, modifying, and displaying string data efficiently.

5) Definition : Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/ Objective:

To study and understand how lists are created and manipulated in Python using indexing, slicing, and list comprehensions for accessing and processing list elements.

Source Code:

```
numbers = [10, 20, 30, 40, 50]
```

Indexing

```
print("Original List:", numbers)
print("First Element:", numbers[0])
print("Third Element:", numbers[2])
```

Slicing

```
print("First Three Elements:", numbers[:3])
print("Last Two Elements:", numbers[-2:])
```

List manipulation

```
numbers.append(60)
numbers.remove(20)
print("After Manipulation:", numbers)
```

List comprehension

```
squares = [x * x for x in numbers]
print("Squares using List Comprehension:", squares)
```

Output:

```
Original List: [10, 20, 30, 40, 50]
First Element: 10
Third Element: 30
First Three Elements: [10, 20, 30]
Last Two Elements: [40, 50]
After Manipulation: [10, 30, 40, 50, 60]
Squares using List Comprehension: [100, 900, 1600, 2500, 3600]
```

Conclusion:

Thus, the program successfully demonstrates how to create and manipulate lists using indexing, slicing, list methods, and list comprehensions in Python.

6) Definition : Write a program to illustrate the use of tuples and sets with basic operations.

Aim/ Objective:

To study and understand the use of tuples and sets in Python and perform basic operations such as accessing tuple elements and adding, removing, and checking elements in a set.

Source Code:

```
# Tuple
numbers = (10, 20, 30, 40, 50)

print("Tuple:", numbers)
print("First Element:", numbers[0])
print("Last Element:", numbers[-1])
print("Tuple Length:", len(numbers))

# Set
fruits = {"Apple", "Banana", "Mango"}
print("\nSet:", fruits)

fruits.add("Orange")
print("After Adding Orange:", fruits)

fruits.remove("Banana")
print("After Removing Banana:", fruits)
print("Is Mango present:", "Mango" in fruits)
```

Output:

```
Tuple: (10, 20, 30, 40, 50)
First Element: 10
Last Element: 50
Tuple Length: 5
```

Set: {'Apple', 'Banana', 'Mango'}

After Adding Orange: {'Apple', 'Banana', 'Mango', 'Orange'}

After Removing Banana: {'Apple', 'Mango', 'Orange'}

Is Mango present: True

Conclusion:

Thus, the program successfully demonstrates the creation and basic operations of tuples and sets in Python. Tuples are used for storing fixed collections of elements, while sets are useful for storing unique elements.

7) Definition : Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/ Objective:

To understand the concept of dictionaries in Python by creating a dictionary, accessing its elements, applying different built-in dictionary methods, and iterating through its key-value pairs.

Source Code:

```
student = {  
    "name": "Shivam",  
    "age": 20,  
    "course": "MCA"  
}  
  
print("Original Dictionary:", student)  
  
# Accessing values  
print("Student Name:", student["name"])  
  
# Dictionary methods  
print("Keys:", student.keys())  
print("Values:", student.values())
```



```
student["city"] = "Rajkot"
print("After Adding City:", student)

student.pop("age")
print("After Removing Age:", student)

# Iteration
print("Dictionary Elements:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
Original Dictionary: {'name': 'Shivam', 'age': 20, 'course': 'MCA'}
Student Name: Shivam
Keys: dict_keys(['name', 'age', 'course'])
Values: dict_values(['Shivam', 20, 'MCA'])
After Adding City: {'name': 'Shivam', 'age': 20, 'course': 'MCA', 'city': 'Rajkot'}
After Removing Age: {'name': 'Shivam', 'course': 'MCA', 'city': 'Rajkot'}
Dictionary Elements:
name : Shivam
course : MCA
city : Rajkot
```

Conclusion:

Thus, the program successfully demonstrates how to create and manipulate a dictionary using different dictionary methods and how to iterate through its key-value pairs in Python.

8) Definition : Write a program to explain mutable and immutable objects in Python.

Aim/ Objective:

To understand the difference between mutable and immutable objects in Python and observe how changes made to these objects affect their values.

Source Code:

```
# Immutable object

a = 10

print("Original value of a:", a)

a = 20

print("Value of a after change:", a)


# Mutable object

numbers = [10, 20, 30]

print("Original list:", numbers)


numbers.append(40)

print("List after adding an element:", numbers)


numbers[0] = 50

print("List after modifying an element:", numbers)
```

Output:

```
Original value of a: 10
Value of a after change: 20
Original list: [10, 20, 30]
List after adding an element: [10, 20, 30, 40]
List after modifying an element: [50, 20, 30, 40]
```

Conclusion:

From this program, it can be observed that immutable objects cannot be changed directly after creation, whereas mutable objects can be modified without creating a new object.

9) Definition : Write a program to define and use user-defined functions with different types of arguments.

Aim/ Objective:

To understand how user-defined functions are created and used in Python and to demonstrate different types of arguments such as positional, keyword, default, and variable-length arguments.

Source Code:

```
# Program to demonstrate user-defined functions with different types of arguments
```

```
# Positional arguments
```

```
def add(a, b):
```

```
    return a + b
```

```
print("Addition:", add(10, 20))
```

```
# Default argument
```

```
def greet(name, message="Welcome"):
```

```
    print(message, name)
```

```
greet("Shivam")
```

```
# Keyword arguments
```

```
def student(name, age):
```

```
    print("Name:", name)
```

```
    print("Age:", age)
```

```
student(age=20, name="Meet")
```

```
# Variable-length arguments
```

```
def total(*numbers):
```

```
    return sum(numbers)
```

```
print("Total:", total(10, 20, 30, 40))
```

Output:

Addition: 30

Welcome Meet

Name: Meet

Age: 20

Total: 100

Conclusion:

The program demonstrates the creation and use of user-defined functions with different types of arguments, making the program more organized, reusable, and flexible.

10) Definition : Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/ Objective:

To understand the concept of recursion in Python by creating a function that repeatedly calls itself to solve a problem, using the calculation of factorial as an example.

Source Code:

```
def factorial(n):
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n - 1)
```

```
num = 5
```

```
print("Number:", num)
print("Factorial:", factorial(num))
```

Output:

Number: 5

Factorial: 120

Conclusion:

The program demonstrates how recursion works in Python by repeatedly calling the same function until the base condition is reached. Here, the factorial of a number is calculated using recursion.